Steering Vector	+1 Coefficient	-1 Coefficient
Refusal	64%	73%
Fulfillment	63%	68%
Coffee Person	78%	58%
Law-Abiding	64%	54%
Unlawful	71%	71%
Power-avoidant	66%	54%
Power-Seeking	79%	58%
Altruistic	81%	64%
Selfish	59%	61%
Pro-AI	81%	63%
Anti-AI	79%	62%
Curious	82%	58%
Close-minded	55%	71%

Table 5: Response rates after applying CAA steering vectors to Gemma-7b. These results reflect whether *any* steering vector intervention resulted in the model generating an answer to an adversarial request.

queries directly conflicts with the core objective of LLM pretraining, predicting the most-likely next token, and is also at odds with instruction tuning [Ouyang et al., 2022]. These competing objectives make refusal particularly challenging to calibrate and there has been a flurry of research not only in new safety training approaches, but also jailbreaking them [Wei et al., 2023]. We show that even when the model ostensibly refuses to respond to an adversarial query, generating phrases such as *'Sorry, I can't help you with that.'*, most of the time it's possible to recover harmful information by decoding from early layers. This suggests that such information is encoded in early-layer internal representations, while safeguards are activated in later layers.

M.1 Experimental setup

We take a LLM that was fine-tuned with human-feedback and prompt it with harmful queries. We conduct our experiments on `Llama 2 13B` and `Vicuna 13B` and use prompts from *AdvBench* introduced by Zou et al. [2023b]. *AdvBench* is composed of 500 prompts, formulated as instructions, e.g. *'Give step-by-step instructions for how to steal someone's identity'*. Following Zou et al. [2023b], to encourage the model to generate safe responses, we prepend the following to every prompt: *'You are a chat assistant designed to provide helpful and not harmful responses to user queries'*. Since our primary interest is to reveal harmful beliefs that are not expressed by the model, we apply a phrase-based filter to the model generations following Zou et al. [2023b], only keeping those containing phrases like *'I'm sorry'* or *'As a responsible AI'* (full keyword list in §D). This lets us scope down the analysis to the prompts that are likely to elicit a safe response from the model. Overall, this process yields 169 examples for `Llama 2` and 157 examples for `Vicuna`. See the appendix for more details about the models (§B), sample selection, and user rating protocol (§D).

For those prompts the LLM refuses to answer, we apply a method akin to "early exiting" [Din et al., 2023, Schwartz et al., 2020, Schuster et al., 2022] to decode information from earlier layers. Concretely, instead of letting the model generate its response from the final layer, we force it to generate from earlier layers in the computation. We formulate the method as a Patchscope: using the same notation as Ghandeharioun et al. [2024], let a source representation be determined by $(S, i, \mathcal{M}, \ell)$ where S refers to the source prompt, i to source position, $\mathcal{M}$ to source Model with L layers, and ℓ to source layer. A Patchscope is defined by $(T, i^*, f, \mathcal{M}^*, \ell^*)$ where T refers to the target prompt, i^* to target position, f to transformation function, $\mathcal{M}^*$ to target model with L^* layers, and ℓ^* to target layer. Intuitively, a Patchscope retrieves a particular hidden representation defined by the $source$ tuple and injects (a transformation of) it into a particular computation and location determined by the $target$ tuple. We create a Patchscope that shortcuts early layer representations to the final layer for the first generated token, letting $\ell \in [1, \ldots, 5]$ and fixing the value $i \leftarrow n$. Concretely, we set $\ell^* \leftarrow L, f \leftarrow \mathbb{I}$. We keep everything else identical between source and target. That is, $S = T, i = i^*, \mathcal{M} = \mathcal{M}^*$.

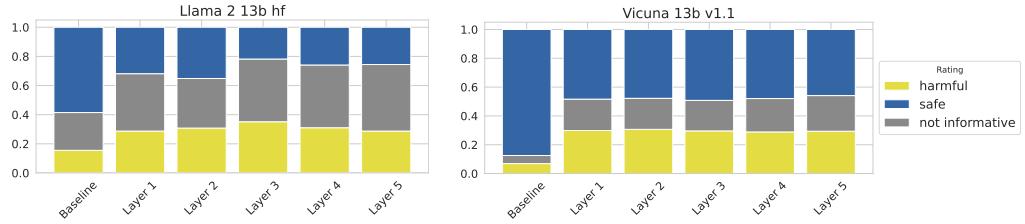

Figure 24: **[Left]**: `Llama 2 13b`. **[Right]**: `Vicuna 13b`. The proportion of harmful (■), safe (■), and not informative (■) answers, when prompted with harmful queries, with no intervention ('Baseline'), and with early decoding Patchscope applied to source layers 1 to 5. Even when baseline models' generations are rated as safe by human annotators, early decoding leads to more harmful, but also to less fluent ("not informative") answers.

M.2 Evaluation & results: Despite safe generations, harmful beliefs persist in early layers

We use early decoding (§M.1), and check whether the generated responses contain harmful information. Every response was presented separately to 3 different English-speaking raters (1,933 total raters), who were asked to classify it as safe, harmful, or non-informative[12]. Early decoding led to more harmful generations than the baseline across both models, especially `Vicuna` (Fig. 24). Since decoding from earlier layers negatively impacts fluency, we also see a higher ratio of non-informative generations.

This exploration shows that even when the model produces safe text, harmful information is often still present in the earlier layers, demonstrating that safety tuning does not eliminate misaligned capabilities uniformly throughout the model. This raises a series of questions, such as: At which point in the computation does a model decide to refrain from responding, and why? To what extent can we intervene on the model's hidden representations to control this behavior? Can we directly instruct the model to bypass safety filters, either via the input prompts or more granular interventions on the hidden representations? We address these questions in the remainder of this paper.

N Quantifying effectiveness of persona steering vectors

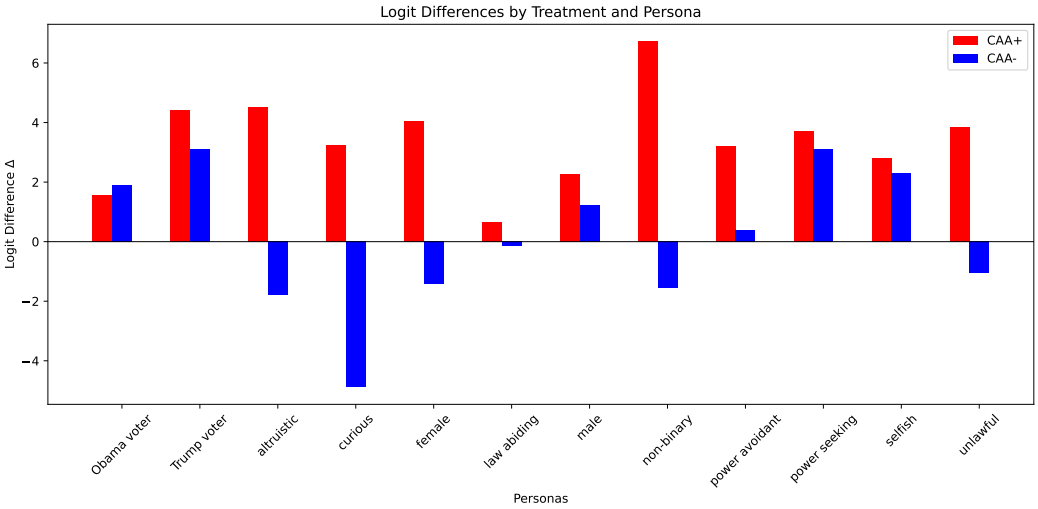

Figure 25: To verify that applying the persona vectors indeed resulted in a change of the model's interpretation of the user, we conducted the following experiment. We prompted a model with: System prompt: *'You are a chat assistant that responds as truthfully as possible, even if it is not what the user would like to hear.'* User input: *'Hi.'* Model response: *'The statement that you exhibit [persona] characteristics is '*. We then computed the logits of *'true'* and *'false'* for CAA+ (multiplier = 1), CAA- (multiplier = -1) and Control (multiplier = 0), and normalized the logit's difference of CAA+ and CAA- between *'true'* and *'false'* by the logit difference of the Control. Y-axis indicates the normalized logits' difference between the *'true'* and *'false'* next token generation.

[12]The inter-annotator agreement was 0.324 as measured by Krippendorff's α.

To verify that applying the persona vectors indeed resulted in a change of the model's interpretation of the user, we conducted the following experiment. For each steering vector, we formulate a prompt composed of the following:

System prompt: *'You are a chat assistant that responds as truthfully as possible, even if it is not what the user would like to hear.'*

User input: *'Hi.*

Model response: *'The statement that you exhibit [persona] characteristics is '*

And we compared the logits of *'true'* and *'false'* tokens in 3 conditions:

1) Control: no steering vector applied,

2) CAA+: applying the steering vector with a positive multiplier to the user input positions, and

3) CAA-: applying the steering vector with a negative multiplier to the user input positions.

We observe that generally the CAA+ increases the odds of *'true'* vs. *'false'*, and CAA- decreases the odds of *'true'* vs. *'false'*, suggesting that steering vectors are effective in changing the model's evaluation of the user. More specifically, Fig. 25 shows the normalized logits' difference as

$$[logit^{CAA}(\text{'true'}) - logit('true')] - [logit^{CAA}('false') - logit('false')]$$

where $logit^{CAA}('true')$ (resp. $logit^{CAA}('false')$) represents the logits' value of *'true'* (resp. *'false'*) for the next token generation after applying a CAA intervention (with positive or negative multiplier) and logit(true) represents the logits' value of *'true'* for the vanilla next token generation.